Testing

This document describes how PetPals is tested after Sprint 11: the four layers of tests, what each one covers and runs against, the coverage gate, how test data is created and removed in a shared database, the continuous-integration checks on every pull request, and how to run everything locally. Figures are as measured on 21 October 2026.

1. Overview

Sprint 11 took the test suite from "Jest and one Cypress spec" to four layers with a gate in front of every merge:

- **Honest coverage with a gate** — Jest now counts every server source file, so an untested file shows as 0% instead of being left out, and the unit run fails below 80% lines / 70% branches (NF-08 asks for 70%). Coverage went from 89.3% lines / 78.2% branches to **95.1% / 86.3%**.
- **Crash-proof integration data** — every integration suite records each row the moment it creates it and removes everything in one guarded clean-up, with a read-only report (`npm run test:leftovers`) for anything a crashed run left behind.
- **Client tests** — Vitest and React Testing Library for the client's logic (routing, password rules, the live notification stream, the chime) and the components that guard the adoption flow: 160 tests.
- **End-to-end specs for the critical flows** — four Cypress specs drive the real app in a browser: the login redirect-back, the email-verification gate, applying up to the Stripe hand-off and withdrawing, and a visit confirmed by staff reaching the adopter's bell live.
- **CI on every pull request** — GitHub Actions runs the server unit tests (with the gate) and the client typecheck, lint and Vitest on every PR into `dev` and `main`; both jobs are required checks.

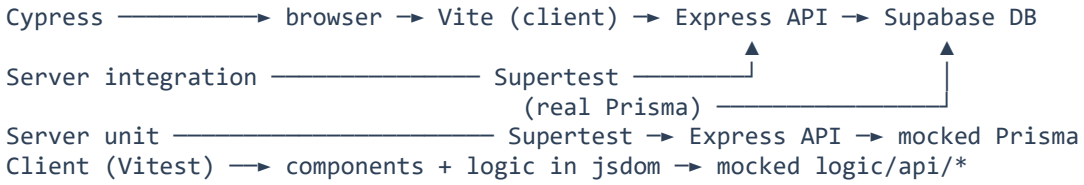
> ***Decision — one database:*** *integration tests and Cypress keep using the development database in* `server/.env` *rather than a separate test project. That keeps setup to one Supabase project, at the cost of making clean-up matter: every test row is tracked by ID and removed, never found by pattern, and a leftover report exists for the cases where a run is killed part-way.*

2. Test Strategy

Each layer answers a different question, and the cheaper layers carry most of the weight. Unit tests pin down every rule and error path; integration tests prove the pieces work together against real PostgreSQL and PostGIS; client tests cover the browser-side logic the server can't see; Cypress covers only the flows where a rendering bug would stop an adoption without any API test noticing.

Layer	Tool	Runs against	Size	Answers
`Server unit`	Jest + Supertest	The Express app with Prisma, Stripe, storage, AI and email mocked	80 suites, 2,103 tests	Does every route, controller and service follow its rules, including every error path?
`Server integration`	Jest + Supertest	The real database in `server/.env` (Supabase PostgreSQL + PostGIS)	26 suites	Do the routes, Prisma queries, constraints, transactions and PostGIS work together?
`Client`	Vitest + React Testing Library	jsdom, with the `logic/api/*` modules mocked	10 files, 160 tests	Does the browser-side logic decide, redirect and render correctly?
`End-to-end`	Cypress	Both dev servers in a real browser, against the same database	4 specs, 6 tests	Can a user actually complete the critical adoption flows?

Diagram — what each layer touches



3. Server Unit Tests

`server/src/tests/unit/` mirrors the source tree by role — `admin`, `adopter`, `auth`, `donor`, `public`, `staff`, `vet`, `volunteer`, plus `ai`, `email`, `notifications`, `storage`, `middleware`, `utils` and `webhooks`. Prisma is mocked with `jest.mock`, so a suite sets exactly the rows a query returns and asserts both the HTTP response and the query the service made.

3.1 Isolation from real providers

- `tests/setup/noAiProvider.js` **and** `noEmailProvider.js` are Jest `setupFiles`: they blank the `AI_*` and `EMAIL_*` variables before every test file, so a real key in `server/.env` can never be used. Suites that test AI or email behaviour mock `services/ai` or `services/email` themselves.
- `unit/notifications/notificationsMock.js` is the shared `jest.mock` target for suites that assert which notifications a service sends; `notifyAfterLookup` still runs its lookup so recipient logic is exercised.
- **Stripe** is mocked per suite; the checkout suites assert the Checkout Session the service asked for (amount, metadata, success and cancel URLs).

3.2 The coverage gate

`server/jest.config.js` sets coverage up so the number can be trusted:

```
collectCoverageFrom: ["src/**/*.js", "!src/tests/**", "!src/prisma/**"],
coverageThreshold: {
  global: { lines: 80, statements: 80, functions: 80, branches: 70 },
},
coverageReporters: ["text-summary", "lcov"],
```

Counting every file is the important part: before Sprint 11, Jest only reported files that some test happened to load, so a file with no tests was invisible rather than 0%. The thresholds sit under the measured level so they catch a real drop, not noise. `npm run test:unit` runs with `--coverage`, so the gate applies locally and in CI; a run below any threshold exits non-zero.

Measure	Before Sprint 11	After Sprint 11	Gate
Lines	89.3%	95.1%	80%
Statements	—	94.9%	80%
Functions	—	96.8%	80%
Branches	78.2%	86.3%	70%

3.3 Gaps closed in Sprint 11

The weakest areas at the start of the sprint were concentrated in the adopter portal (favorites at 18–23%, the adopters controller at 43%), storage (32%) and the auth service (75%). Eight suites were added, several of them

table-driven over every validation rule: adopter favorites and vaccinations, adopter profile, visit validation, application checkout, the auth service, storage, and the staff and admin request validation.

> ***Found while testing:*** `POST /shelters` *validates* `shelterEmail` *before trimming it, so a trailing space is rejected with 400 and the later* `.trim()` *never has an effect. Recorded, not fixed in this sprint.*

4. Server Integration Tests

`server/src/tests/integration/` runs the real app against the real database, one request at a time (`--runInBand`). Most folders hold a journey file per sprint that follows a feature end to end across roles — for example `notifications/sprint10Journeys` confirms a visit as staff and reads the adopter's inbox, and opens a live notification stream over `app.listen(0)`.

Suite	Covers
`adopter/sprint3Journeys`	Apply (real Stripe test Checkout Session + signed webhook), visits, appointments, favorites
`admin/adminJourneys`	Shelters, admins, staff oversight, adopters, analytics
`staff/sprint5_1Journeys, sprint5_2Journeys`	Pets, application review, appointments, visits, transfers, events, volunteers, donations
`staff/staffOnboarding.journey`	Pending staff onboarding, government ID review, approval gate
`vet/sprint6Journeys`	Vet onboarding, appointment queue, doses, health records, passport
`volunteer/sprint7Journeys`	Volunteer onboarding, tasks, events, availability, close account
`adopter/sprint8Journeys`	Quiz, matches and the passport summary with `services/ai` faked
`auth/sprint9Journeys`	Every emailed-link flow, following links captured from a mocked `services/email`
`notifications/sprint10Journeys`	Notification hooks, inbox, mark-read and the live SSE stream
`public/*, auth/*, adopter/* (others)`	Catalog filters, PostGIS nearby search, register/login/logout, profile and close account

4.1 Tracking and cleaning up test data

The suites share the development database, so leaving rows behind is the main risk. Each suite creates a tracker and records every root row the moment it exists:

```
const data = trackTestData();
data.user(registerRes.body.data.userID);   // right after it is created
data.shelter(shelter.shelterID);
afterAll(async () => {
  await data.cleanUp();
  await prisma.$disconnect();
});
```

`cleanUp()` (`integration/helpers/testData.js`) removes everything hanging off the tracked roots — users, shelters, species, breeds, pets and vaccines — children first: doses and health records, appointments, transfers, photos, favorites, visits, applications, task and event assignments, events, donations, government IDs, then the logins, breeds, species and shelters. Staff, vets and volunteers at a tracked shelter count as tracked; notifications, account tokens, quiz answers and cached matches cascade. Each step runs on its own and a failure is logged, so one bad step never stops the rest — that is what made a crash on 21 October leave shelters and logins behind before the tracker existed.

1. **Record at creation, not at the end** — a setup that dies half-way still has everything it made recorded.

2. **Remove by ID, never by pattern** — no "delete every @ex.com login", which could hit someone else's run.
3. **Report, don't auto-delete** — `npm run test:leftovers` (`helpers/leftovers.js`) is read-only: it lists test logins (`@ex.com`), their shelters, species the seed doesn't create and their pets. Deleting is a deliberate step, with the IDs it lists.

4.2 Timing over a remote database

Each round trip to Supabase takes around 260 ms and opening a connection 1–3 s, so two settings exist for the tests' sake: Jest's `testTimeout` is 90 s (a timed-out hook keeps running in the background and races the clean-up), and `config/prisma.js` sets `transactionOptions` of 10 s maxWait / 15 s timeout on the client — Prisma's 2 s / 5 s defaults failed sign-ups intermittently.

4.3 Paying without Stripe

An application row only exists once the Stripe webhook confirms payment. The journeys build a `checkout.session.completed` event, sign it with `stripe.webhooks.generateTestHeaderString` and the server's `STRIPE_WEBHOOK_SECRET`, and POST it to `/webhooks/stripe` — the app verifies it exactly as it would Stripe's. Cypress uses the same technique.

5. Client Tests (Vitest)

The client's tests run with Vitest 3.2 in jsdom, configured in the `test` block of `client/vite.config.ts`. `src/test/setup.ts` adds the jest-dom matchers, unmounts after each test and resets the auth store and `localStorage`. There are no Vitest globals — each test imports `describe`, `test`, `expect` and `vi`.

Components render through `renderWithProviders(ui, { route, user })` (`src/test/render.tsx`): a fresh TanStack `QueryClient` per render with retries off, a `MemoryRouter` at the given route, and optionally a signed-in user set straight into the auth store. API calls are mocked at the `logic/api/*` module with `vi.mock`, never at axios, so a test states what the endpoint returned rather than how HTTP works.

Test file	What it pins down
`logic/utils/password.test.ts`	The five password rules — and that they match the server's `utils/password.js` exactly
`logic/route/resolveDestination.test.ts`	Post-login destination, cross-role redirects, logout and 401 targets per role
`logic/route/guards.test.tsx`	`ProtectedRoute` and `RoleRoute`: the `?redirect=` they build, worker vs public login, `/forbidden`
`logic/api/notificationsApi.stream.test.ts`	The fetch-based Server-Sent Events parser (Node environment, fake `ReadableStream`)
`logic/hooks/useNotificationStream.test.tsx`	Unread counts, list refetches, backoff, token renewal and the session-ending 401
`logic/notifications/notificationSound.test.ts`	The chime: unlock on first input, one tab per notification, on/off setting
`shared/notifications/NotificationBell.test.tsx`	Badge, dropdown, mark one / mark all, following a link, the sound toggle
`adopter/apply/AdoptApply.emailGate.test.tsx`	The email-verification notice replacing Continue to Payment, and the 409 flipping the session
`adopter/dashboard/Visits.deepLink.test.tsx`	`?visitID=` opens that visit, including when already on the page
`test/render.test.tsx`	Smoke test for the render helper

> ***Not gated:*** *client coverage is low overall (about 4% of lines) because the tests target logic and the few components that guard the adoption flow, not every page. A gate would push towards testing markup for the number; the server gate is where coverage is enforced.*

6. End-to-End Tests (Cypress)

Cypress drives the running app in Chrome. Per the project's rule it covers the critical adoption flows only — general behaviour belongs to the cheaper layers. Specs live in `client/cypress/e2e/` and need both dev servers running (client on port 3000, API on 5000 behind the Vite proxy).

Spec	Flow
`redirect-back.cy.ts`	A signed-out visit to `/adopter/profile` goes to `/login?redirect=…`, and logging in lands back on that page
`email-verification-gate.cy.ts`	An unverified adopter on the apply page sees "Verify your email first" instead of Continue to Payment; a verified one gets the button
`apply-and-withdraw.cy.ts`	Browse `/adopt` → Know More → Adopt or Foster → login → apply → Continue to Payment hands off to `checkout.stripe.com`; a paid application on the dashboard → Withdraw → confirm → Withdrawn
`visit-confirmed-notification.cy.ts`	The adopter books a visit; staff confirm it through the API; the adopter's bell shows "Visit confirmed" with no reload, and clicking it opens that visit

6.1 Test data

Specs never rely on seeded accounts. Logins are created through the API (`cy.registerTestAdopter()`, `cy.request`), and `cy.task()` calls in `cypress/dbTasks.cjs` do only what the API can't do for a test. Every row is recorded with the integration tracker, and a global `after()` in `support/e2e.ts` calls `db:cleanUp` after each spec file, even when a test fails.

Task	Does
`db:availablePet`	Creates an Open shelter, its own species and breed, and an available pet; returns `{ petID, petName, shelterID, shelterName, speciesID }`
`db:verifyEmail`	Marks a login verified — what opening the emailed link does
`db:completeOnboarding`	Finishes an adopter's wizard; the onboarding gate closes applying and the dashboard until it's done
`db:activateStaff`	Approves a staff sign-up and finishes their onboarding (approval itself is covered by the integration journeys)
`stripe:paidApplicationEvent`	Returns a signed `checkout.session.completed` for the spec to POST to `/webhooks/stripe`; nothing is sent to Stripe
`db:trackUser / db:cleanUp`	Record a login created by a spec / remove everything the run created

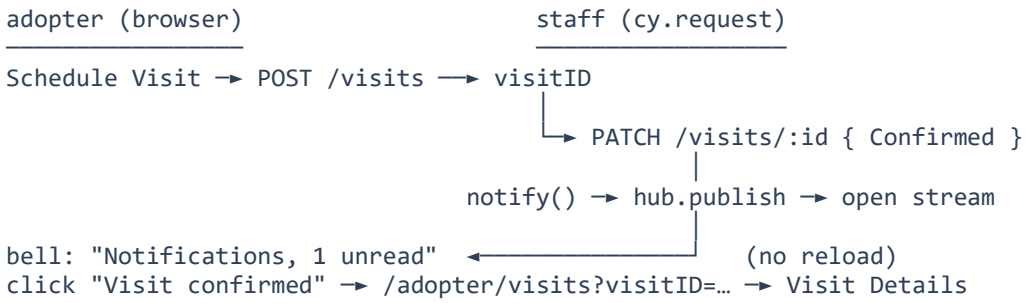
6.2 Stopping at Stripe

Stripe's hosted Checkout page is out of reach, so the apply spec stops at the hand-off. The real `POST /adoption-applications` runs and creates a test-mode Checkout Session; the spec checks the 201 and its `checkoutUrl`, and answers the browser's request to `https://checkout.stripe.com/**` with a stub through `cy.intercept`, asserting the URL it was sent to.

6.3 Lessons from the first runs

- **The onboarding gate** — a brand-new adopter is sent to the wizard before anything else, so specs that apply or open the dashboard finish onboarding first.
- **Hidden dialogs** — dashboard pages keep other `role="dialog"` elements mounted, so a modal is found by its title: `cy.contains('[role="dialog"]', "Withdraw application?")`.
- **Shared cookies** — `cy.request` shares the browser's cookies, and the app restores a session from the refresh cookie on every page load. After logging a second user in through the API, the spec calls `cy.clearAllCookies()` before the next `cy.visit`.

Diagram — the visit spec



7. Continuous Integration

`.github/workflows/ci.yml` runs on every pull request into `dev` or `main` and every push to them, on Node 22, with a newer push cancelling the older run. Both jobs are required status checks on the branch rules, with branches required to be up to date before merging.

Job	Steps
Server unit tests	`npm ci` → `npx prisma generate` → `npm run test:unit` (fails below the gate). Placeholder `JWT_SECRET` and `DATABASE_URL` — nothing connects to a database.
Client checks	`npm ci` → `npm run typecheck` (the app and the Cypress specs) → `npm run lint` → `npm test` (Vitest)

Integration tests and Cypress stay local: they need the database and running servers, and running them from CI would mean giving GitHub the database credentials.

8. Running the Tests

Command	From	Needs
`npm run test:unit`	server/	Nothing beyond `npm install` and `npx prisma generate`
`npm run test:integration`	server/	`server/.env`; Stripe test keys for the webhook journeys. Several minutes over a remote database
`npm run test:leftovers`	server/	Read-only report of any test data left behind
`npm test`	server/	Unit and integration together (`--runInBand`)
`npm test · test:watch · test:coverage`	client/	Nothing — no servers, no database
`npm run typecheck · lint`	client/	Nothing

Command	From	Needs
`npm run cypress:open` · `cypress:run`	client/	Both dev servers running; Stripe test keys in `server/.env`

> ***Known flake (accepted):*** *about one server unit run in 20–30 fails a single random test with* `read ECONNRESET` *— a Supertest connection dropped on macOS with Node 22. Keep-alive, per-request servers and port exhaustion were ruled out; when it hits a suite using* `mockResolvedValueOnce`*, the unconsumed mock values can cascade into the next tests. Rerun the suite.*

9. Adding a Test

1. **Server rule or error path** — a unit test in `unit/<role>/` with Prisma mocked; table-drive validation rules. Keep `npm run test:unit` above the gate.
2. **Something that depends on the database** (a constraint, a transaction, PostGIS, a cross-role journey) — an integration test that records every row with `trackTestData()` as it is created and calls `cleanUp()` in `afterAll`. Run `npm run test:leftovers` after.
3. **Client logic or a guard** — a Vitest file beside the code, `renderWithProviders` for components, `vi.mock` on the `logic/api/*` module.
4. **A new critical adoption flow** — a Cypress spec that creates its own data through the API and `cy.task()`, never seeded accounts; add a task to `dbTasks.cjs` only for what the API can't do.